uv



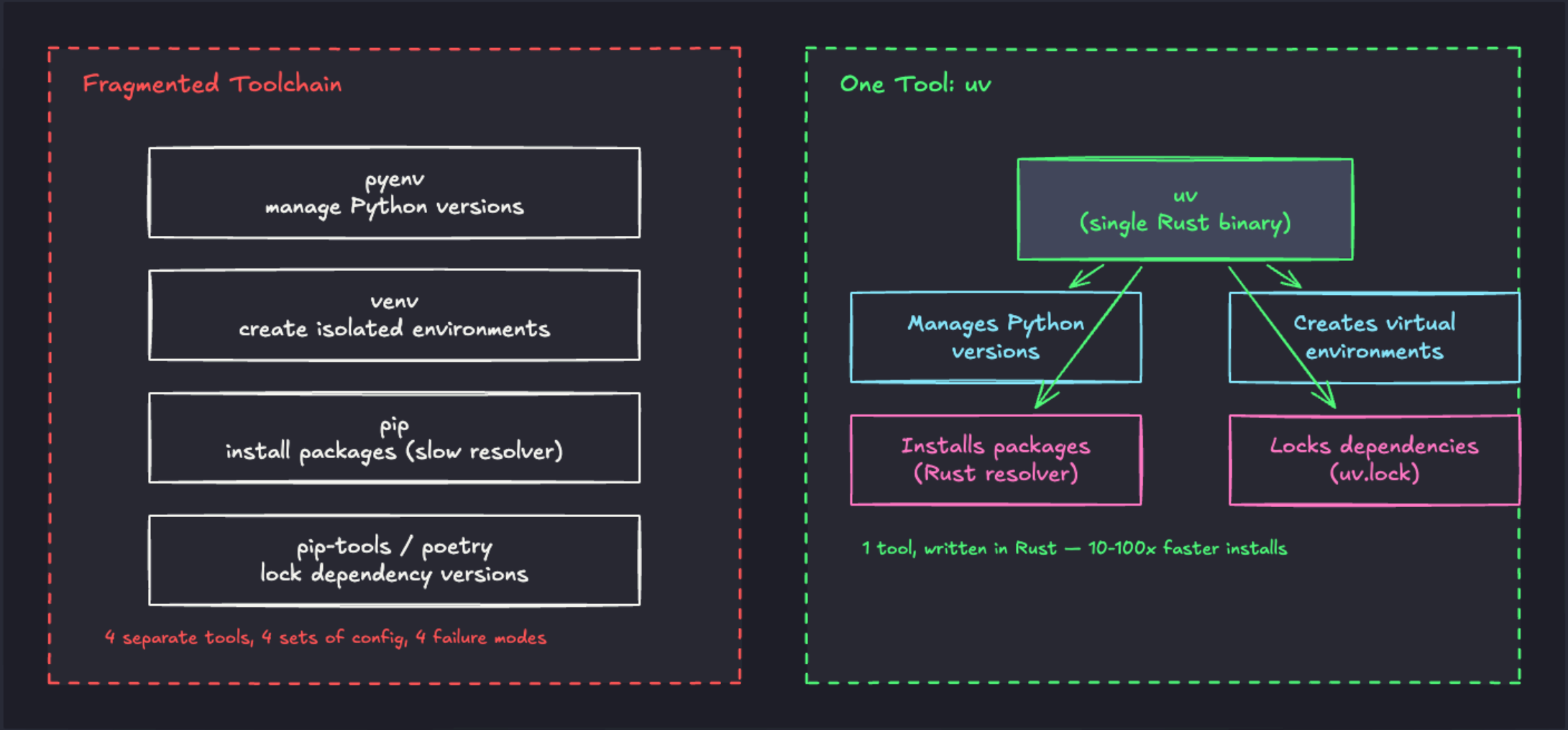
© 2026 SophiArch lms.sophiarch.com

The Problem: Too Many Tools

- **pyenv** to manage Python versions
- **venv** to create isolated environments
- **pip** to install packages — with a notoriously slow dependency resolver
- **pip-tools** or **poetry** on top, just to get reliable version locking
- Four tools, four config formats, four things that can go out of sync

© 2026 SophiArch lms.sophiarch.com

Fragmented Toolchain vs. uv



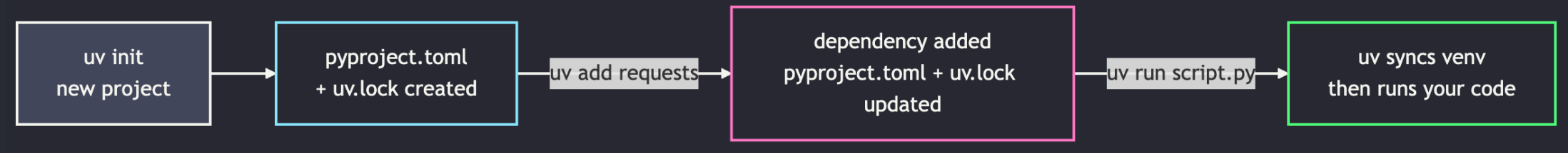
© 2026 SophiArch lms.sophiarch.com

What `uv` Actually Is

- A single, fast, all-in-one Python package and project manager, written in Rust
- Built by **Astral** — same team behind the `ruff` linter/formatter
- Drop-in replacement for `pip`, `venv`, `pip-tools`, `virtualenv`, and (largely) `pyenv`
- Manages `pyproject.toml` and `uv.lock` automatically — no manual `requirements.txt` bookkeeping

© 2026 SophiArch lms.sophiarch.com

How the Workflow Flows



© 2026 SophiArch lms.sophiarch.com

Installation

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Or via pip, if you already have Python installed
pip install uv
```

- Installs a single self-contained binary — no Python required to bootstrap it
- Verify with `uv --version`

© 2026 SophiArch lms.sophiarch.com

Demo: Start a New Project

```
# 1. Scaffold a new project
uv init my-project
cd my-project

# 2. Add a dependency — creates the venv automatically if needed
uv add requests

# 3. Run code inside the managed environment — no activate step
uv run python -c "import requests; print(requests.__version__)"
```

© 2026 SophiArch lms.sophiarch.com

Demo: Everyday Commands

```
# Add a dev-only dependency (e.g. a test runner)
uv add --dev pytest

# Remove a dependency
uv remove requests

# Recreate the exact environment from uv.lock (e.g. on a teammate's machine)
uv sync

# Run a script or command inside the project's environment
uv run pytest
```

© 2026 SophiArch lms.sophiarch.com

uv vs. Traditional Commands

Traditional	uv equivalent
`python -m venv .venv`	*(automatic — created on first `uv add` / `uv run`)*
`source .venv/bin/activate`	*(not needed — use `uv run ...`)*
`pip install requests`	`uv add requests`
`pip freeze > requirements.txt`	*(automatic — `uv.lock` stays in sync)*
`pip install -r requirements.txt`	`uv sync`
`pyenv install 3.12`	`uv python install 3.12`

© 2026 SophiArch lms.sophiarch.com

Common Pitfalls

- **Editing `pyproject.toml` by hand for dependencies** — use `uv add` / `uv remove` so `uv.lock` stays in sync
- **Forgetting `uv run`** — running `python script.py` directly uses whatever Python is on your PATH, not the project's venv
- **Committing `.venv/` but not `uv.lock`** — it's the reverse of what should be committed; `.venv/` is disposable, `uv.lock` is the source of truth
- **Mixing `pip install` into a uv-managed project** — installs outside `uv.lock` tracking and can drift from what teammates have

© 2026 SophiArch lms.sophiarch.com

Cheat Sheet

Action	Command
Install uv	`curl -LsSf https://astral.sh/uv/install.sh | sh`
New project	`uv init <name>`
Add a dependency	`uv add <package>`
Add a dev dependency	`uv add --dev <package>`
Remove a dependency	`uv remove <package>`
Sync environment to lock file	`uv sync`
Run code in the project env	`uv run <command>`
Install a Python version	`uv python install <version>`

© 2026 SophiArch lms.sophiarch.com

References

Topic	Source
Official `uv` documentation	Astral. (2026). uv — An extremely fast Python package manager. Astral Docs.
`uv` GitHub repository	Astral. (2026). astral-sh/uv. GitHub.
`uv` benchmark results	Astral. (2026). uv: Benchmarks. GitHub.

© 2026 SophiArch lms.sophiarch.com

Recap & Practice

- `uv` replaces pyenv + venv + pip + pip-tools/poetry with one fast, unified tool
- `uv add`, `uv run`, and `uv sync` cover nearly everything you'll need day to day
- **Before your next assignment: `uv init` the project instead of `python -m venv`**

© 2026 SophiArch lms.sophiarch.com